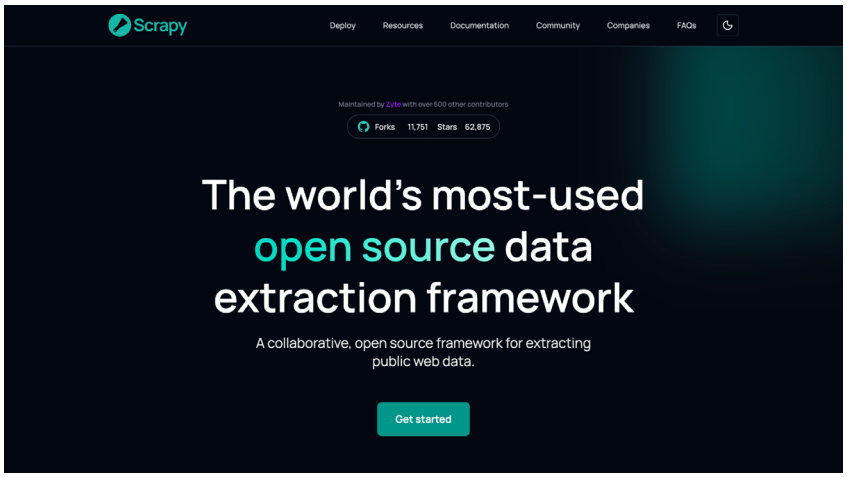

The unit of work in Scrapy is a spider, a class that says where to start and what to do with each response. Here is an example against another sandbox site, *quotes.toscrape.com*, which paginates through a few hundred quotes.

```
import scrapy
class QuotesSpider(scrapy.Spider):
    name = "quotes"
    start_urls = ["https://quotes.
toscrape.com/page/1/"]
    def parse(self, response):
        for quote in response.css("div.
quote"):
            yield {
                "text": quote.css("span.
text::text").get(),
                "author": quote.
css("small.author::text").get(),
                "tags": quote.css("div.
tags a.tag::text").getall(),
            }
        next_page = response.css("li.
next a::attr(href)").get()
        if next_page is not None:
            yield response.follow(next_
page, callback=self.parse)
```

A lot happens in these few lines. Each dictionary you *yield* becomes a scraped item that Scrapy routes through its output machinery. The *response.follow* call at the end is the crawling loop: it finds the 'next' link, resolves it even though it is relative, and schedules another request that runs *parse* again on the next page. You never write a *while* loop or track which pages you have visited. Save the file and run it with a single command:

```
scrapy runspider quotes_spider.py -O
quotes.json
```

When I ran exactly this, Scrapy walked all ten pages and wrote 100 items to *quotes.json* in under five seconds, following pagination nine links deep without any book-keeping from me (Figure 3). For free, and without appearing in the spider code, you also get asynchronous requests handled concurrently, automatic retries on transient failures, auto-throttling so you do not overwhelm the target, and item pipelines where you can validate, clean, or store records as they stream through.

Scrapy's flexibility comes largely from its middleware layers, and there are two kinds. Downloader middlewares sit between the engine and the network, so they are where request and response handling gets customised: proxies, retry

```
zsh — scrapy crawl

$ scrapy runspider quotes_spider.py -O quotes.json
[scrapy.core.engine] INFO: Spider opened
[scrapy.core.engine] DEBUG: Crawled (200) <GET https://quotes.toscrape.com/page/1/> (referer: None)
[scrapy.core.engine] DEBUG: Crawled (200) <GET .../page/2/> (referer: .../page/1/)
[scrapy.core.engine] DEBUG: Crawled (200) <GET .../page/3/> (referer: .../page/2/)
                                ... pages 4 through 9 ...
[scrapy.core.engine] DEBUG: Crawled (200) <GET .../page/10/> (referer: .../page/9/)
[scrapy.extensions.feedexport] INFO: Stored json feed (100 items) in: quotes.json
[scrapy.statscollectors] INFO: Dumping Scrapy stats:
 'elapsed_time_seconds': 4.80,
 'item_scraped_count': 100,
 'response_received_count': 10,
[scrapy.core.engine] INFO: Spider closed (finished)
# 100 quotes across 10 pages, following pagination, in under five seconds
```

Figure 3: The same spider run end to end. Scrapy follows the 'next' link from page to page and reports 100 items scraped across 10 responses, with no pagination bookkeeping in the spider